Chest X-ray Pneumonia Detection System

In-Depth Project Explanation

A complete, plain-language walkthrough of how everything works: the AI pipeline and the web application.

Team 19 - School of CSAI, Zewail City of Science and Technology - Supervisor: Prof. Dr. Khaled Mostafa - June 2026

How to read this document: it is written so that someone who has never seen the project can finish it and confidently explain every part. It has two main sections. Section A covers the AI pipeline (the data, the models, how they were trained and evaluated, and why we got the results we did). Section B covers the web application (the frontend, backend, database, security, and how a request flows end to end). Each section ends with a bank of questions the examiners are likely to ask, with answers.

Table of Contents

Section A - The AI Pipeline

In one sentence: we take a chest X-ray image, pass it through a trained deep-learning model, and get back either a probability that the lungs show pneumonia (classification) or a box around the pneumonia region with a confidence score (detection). Everything below explains how we get from a raw medical image to that answer, and why it works.

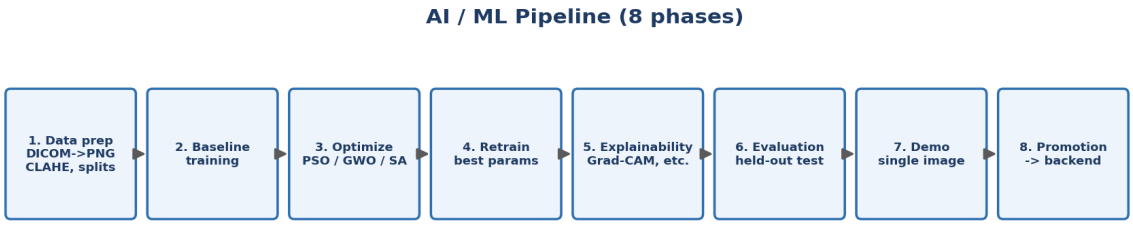

Figure A1. The eight phases of the AI pipeline.

A1. The dataset: RSNA Pneumonia Detection Challenge

We used the RSNA Pneumonia Detection Challenge dataset, published by the Radiological Society of North America. It contains about 26,684 frontal chest X-rays, each labeled by radiologists. Crucially, it provides two kinds of labels: an image-level label (pneumonia or not) that we use for classification, and bounding boxes (the exact rectangles where lung opacity consistent with pneumonia appears) that we use for detection. We chose it because it is large, professionally annotated, widely used as a benchmark (so our numbers can be compared to others), and supports both tasks at once.

Why this matters: the quality of a model is capped by the quality of its data. Because RSNA is expert-labeled and balanced enough to be usable, our results reflect the model, not label noise we introduced ourselves.

A2. From a raw scan to a model input (preprocessing)

Medical scans do not arrive as ordinary images. They come as DICOM files, a medical format that stores the pixel data plus patient metadata. Our preprocessing turns each DICOM into a clean PNG the model can read, in these steps:

- Read the DICOM with the pydicom library to get the raw grayscale pixel array.
- **Robust intensity normalization:** X-ray brightness varies between machines, so we clip each image to its 2nd-98th intensity percentiles and rescale to 0-255. This removes extreme outliers and makes images comparable.
- **CLAHE contrast enhancement:** Contrast Limited Adaptive Histogram Equalization (clip limit 2.0, 8x8 tiles) locally boosts contrast so subtle lung opacities become visible. It equalizes brightness in small tiles rather than globally, which is ideal for X-rays.
- Resize to a fixed size (224x224 for classifiers, 640x640 for detectors) and save the original dimensions so detection boxes can be scaled back correctly.

The most important lesson here (a likely exam question): our early results were suspiciously high. The cause was a data leak: contrast enhancement had been applied differently to pneumonia and normal images, so the model could 'cheat' by detecting the preprocessing instead of the disease. We fixed it by applying the exact same CLAHE to every image regardless of label. The honest, lower numbers we now report are the real ones. This is the single most valuable thing we learned: trust the protocol, not the number.

A3. Splitting the data correctly (no leakage)

We split the data into training, validation, and test sets at the patient level, not the image level, with a fixed random seed (42) for reproducibility. A single patient can have several X-rays; if some of a patient's images were in training and others in testing, the model could memorize that specific patient and score artificially high. Splitting by patient guarantees the test set contains people the model has never seen. The split is also stratified, meaning the pneumonia-to-normal ratio is kept the same in each part. The held-out test set is 20 percent of patients: 4,135 normal and 1,202 pneumonia images. It is never touched during training or model selection.

A4. Two tasks: classification and detection

Classification answers 'is there pneumonia anywhere in this image?' with a probability. Detection answers 'where is it?' by drawing one or more boxes, each with a confidence. We built both because doctors want both a yes/no screen and a visual pointer to the suspicious region. The deployed default is a detector (Faster R-CNN) because localization is more useful clinically.

A5. The models, explained from scratch

All five models use transfer learning: instead of learning vision from zero, they start from weights pretrained on ImageNet (a huge natural-image dataset) and are then fine-tuned on chest X-rays. This is standard in medical imaging because medical datasets are small, and the low-level features (edges, textures) learned on ImageNet transfer well.

The three classifiers

- **ResNet50:** a 50-layer convolutional network whose key idea is the 'residual connection', a shortcut that lets the signal skip layers. This solves the problem that very deep networks become hard to train, and made very deep networks practical.
- **DenseNet121:** every layer is connected to every later layer, so features are reused throughout the network. It is parameter-efficient and was the backbone of CheXNet, the famous radiologist-level pneumonia model.
- **EfficientNet-B0:** uses 'compound scaling' to balance network depth, width, and input resolution. It is our best classifier and also by far the smallest (4 million parameters), which is why we consider it the most efficient choice.

The two detectors

- **Faster R-CNN (two-stage):** first a Region Proposal Network suggests candidate boxes, then a second head classifies each box and refines its coordinates. It uses a ResNet50 backbone with a Feature Pyramid Network so it sees objects at multiple scales. It is accurate, especially in recall, which is why we deploy it.
- **YOLOv8 (one-stage):** predicts all boxes in a single forward pass with an anchor-free head. It is much faster and smaller, but in our tests it missed many more pneumonia regions (lower recall) than Faster R-CNN.

A6. How we trained the classifiers (every parameter)

Setting	Value	Why
Optimizer	Adam	Adaptive, robust default for fine-tuning

Setting	Value	Why
Learning rate	1e-4	Small, so pretrained weights are not destroyed
Loss	Cross-entropy, class-weighted	Weights the rare pneumonia class so it is not ignored
LR scheduler	ReduceLROnPlateau (val AUC, x0.5)	Lowers LR when validation stops improving
Mixed precision (AMP)	On (GPU)	Faster training, less memory
Early stopping	Patience 4 on val AUC	Stop before overfitting; restore best weights
Augmentation (train only)	h-flip, rotate 10, color jitter	Teaches invariance, reduces overfitting
Normalization	ImageNet mean/std	Matches the pretrained backbone
Image size	224 x 224	Standard for these backbones

Two details worth highlighting. **Class weighting:** pneumonia is the minority class, so we weight the loss by inverse class frequency; without this, a model can score high accuracy by always predicting 'normal'. **Best-checkpoint restore:** we keep the weights from the epoch with the best validation AUC, not the last epoch, so we never ship an overfitted model.

A7. How we trained the detectors

Faster R-CNN was trained for up to 10 epochs (learning rate 5e-3, batch size 4, weight decay 5e-4), freezing the backbone for the first epoch so the new detection heads stabilize before the whole network is fine-tuned. It optimizes the standard Faster R-CNN multi-task loss: the proposal network's objectness and box losses plus the final head's classification and box-regression losses. At inference we apply Non-Maximum Suppression to remove duplicate overlapping boxes (explained in Section B). YOLOv8 was trained through the Ultralytics pipeline with its built-in augmentation.

A8. Hyperparameter optimization: PSO, GWO, SA

To search for better settings we implemented three nature-inspired optimizers. They are all ways to search a space of configurations without trying every combination:

- **Particle Swarm Optimization (PSO):** a swarm of candidate solutions 'flies' through the search space, each pulled toward its own best result and the swarm's best, like birds flocking to food.
- **Grey Wolf Optimizer (GWO):** models a wolf pack hierarchy; the best three solutions ('alpha, beta, delta') lead the rest of the pack toward promising regions.
- **Simulated Annealing (SA):** inspired by cooling metal; it sometimes accepts worse solutions early (high 'temperature') to escape local traps, then settles as it cools.

Honest result (a likely exam question): we scored each candidate with a cheap proxy (a few epochs on part of the data) so the search fit in one GPU session. That proxy was too noisy: the settings it preferred did not survive a full retrain. So we kept the validation-best checkpoint per model instead of the search-suggested one. The lesson, that a lightweight proxy search does not automatically beat a carefully tuned baseline, is a real and defensible finding, not a failure.

A9. Evaluation metrics, explained simply

- **Accuracy:** fraction of all predictions that are correct. Misleading on imbalanced data, so we do not rely on it alone.
- **Recall / Sensitivity:** of the truly sick patients, how many we caught. This is the most important metric here, because a missed pneumonia is dangerous.
- **Specificity:** of the truly healthy patients, how many we correctly cleared.
- **Precision:** of the patients we flagged, how many really had pneumonia.
- **F1:** the balance (harmonic mean) of precision and recall.
- **AUC:** the area under the ROC curve; the probability the model ranks a random sick patient above a random healthy one. 0.5 is random, 1.0 is perfect. It is threshold-independent, which is why we use it as our headline classification metric.
- **Confusion matrix:** the table of true/false positives and negatives that all the above are computed from.
- **mAP, IoU, mAP@0.5:** for detection. IoU (Intersection over Union) measures box overlap; a box counts as correct if its IoU with the truth exceeds a threshold (e.g., 0.5). mAP is the average precision across recall levels; mAP@0.5 uses a 0.5 IoU threshold, and mAP@[.5:.95] averages over stricter thresholds.

A10. Results, why we got them, and how to improve

Model	Key metric	Value
EfficientNet-B0 (best classifier)	AUC	0.886
DenseNet121	AUC	0.883
ResNet50	AUC	0.884
Faster R-CNN (deployed detector)	Recall / mAP@0.5	0.812 / 0.381
YOLOv8n	Recall / mAP@0.5	0.382 / 0.346

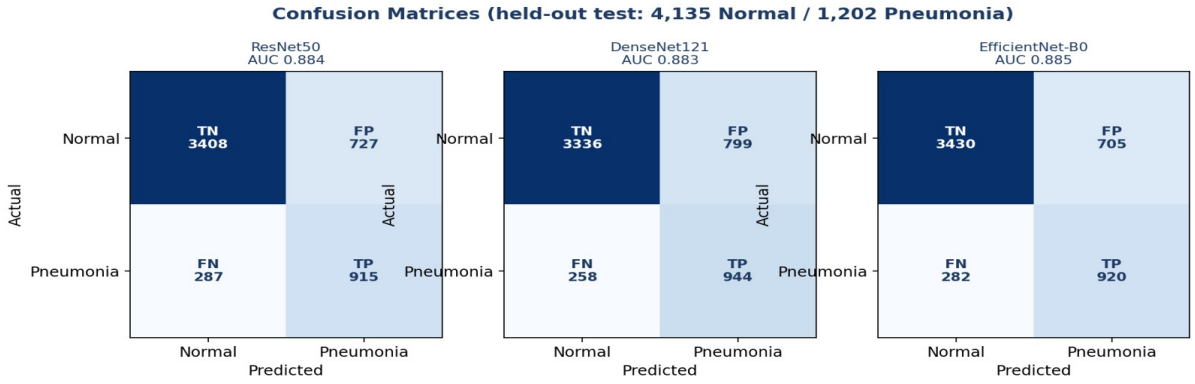

Figure A2. Confusion matrices for the three classifiers.

Why these numbers? An AUC around 0.88 is what honest pneumonia classifiers reach on this kind of data; for comparison, the well-known CheXNet reported 0.768 on a different dataset. The classifiers keep specificity high (around 0.83) while catching about 76-79 percent of pneumonia. For detection, both detectors reach a similar mAP@0.5 (about 0.35-0.38, exactly the 0.32-0.39 range reported in the literature), but Faster R-CNN recalls 0.812 of pneumonia regions versus 0.382 for YOLO. Because missing a case is the costly error, we deploy Faster R-CNN even though it is larger and slower.

How to get better results: train on more and more diverse data (different hospitals and machines); use higher input resolution so faint opacities survive; ensemble several models; tune the decision threshold to trade specificity for recall as a clinic prefers; add stronger, medically realistic augmentation; pretrain on a large chest-X-ray corpus before fine-tuning; and, for detection, train longer with the full (not proxy) objective. External validation on a second dataset would also make the results more trustworthy.

A11. Explainability: showing the doctor why

Every prediction is shown with a heatmap so the clinician can check that the model looked at the lungs, not at text or an artifact. We implemented several methods:

- **Grad-CAM, Integrated Gradients, GradientSHAP, Score-CAM (for classifiers):** these highlight the pixels that most influenced the decision, using gradients or by perturbing the image.
- **Eigen-CAM and occlusion sensitivity (for detectors):** standard gradient saliency does not apply cleanly to detectors, so we use Eigen-CAM (which analyzes the network's activations) and occlusion (sliding a mask over the image and watching how the prediction drops).

Explainability is rendered live with each request and is best-effort: if a method fails, it is skipped rather than breaking the prediction.

A12. Section A - Questions the examiners may ask

Q: Why did your early results look almost perfect, and what did you do?

A: They were inflated by a preprocessing data leak: contrast enhancement was applied differently to the two classes, so the model detected the preprocessing, not the disease. We fixed it by applying identical CLAHE to every image and we report the honest, lower numbers.

Q: Why split by patient instead of by image?

A: Because one patient can have multiple X-rays. Splitting by image could put the same patient in both train and test, letting the model memorize that patient and score artificially high. Patient-wise splitting guarantees the test set is truly unseen.

Q: Why is recall your most important metric?

A: Because in pneumonia screening a false negative (a missed case) can be life-threatening, while a false positive only leads to a second look. We optimize and select models to minimize missed cases.

Q: Why deploy Faster R-CNN instead of the faster YOLO?

A: Both reach similar mAP@0.5, but Faster R-CNN recalls 0.812 of pneumonia regions versus YOLO's 0.382. We accept the larger, slower model because catching cases matters more than speed here.

Q: How did you handle class imbalance?

A: We weighted the loss by inverse class frequency so the rare pneumonia class is not ignored, and we evaluate with AUC, recall, and the confusion matrix rather than accuracy alone.

Q: How do you prevent overfitting?

A: Transfer learning, train-only augmentation, weight decay, a learning-rate scheduler, early stopping on validation AUC, and restoring the best-validation checkpoint rather than the last epoch.

Q: Did the metaheuristic optimization help?

A: Not on our cheap proxy: PSO, GWO, and SA preferred settings that did not survive a full retrain. We therefore selected the validation-best checkpoint. The honest takeaway is that proxy search does not automatically beat a tuned baseline.

Q: Is the model safe to use clinically?

A: It is decision support, not an autonomous diagnosis. The doctor confirms every result, and every prediction comes with a confidence and an explainability heatmap so it can be challenged. It would need external validation and regulatory approval before clinical use.

Section B - The Web Application Pipeline

The application has three tiers: a frontend the doctor sees, a backend that does the work, and a database that stores everything. A dedicated inference service inside the backend runs the AI models. This section explains each piece and then traces a single request from click to result.

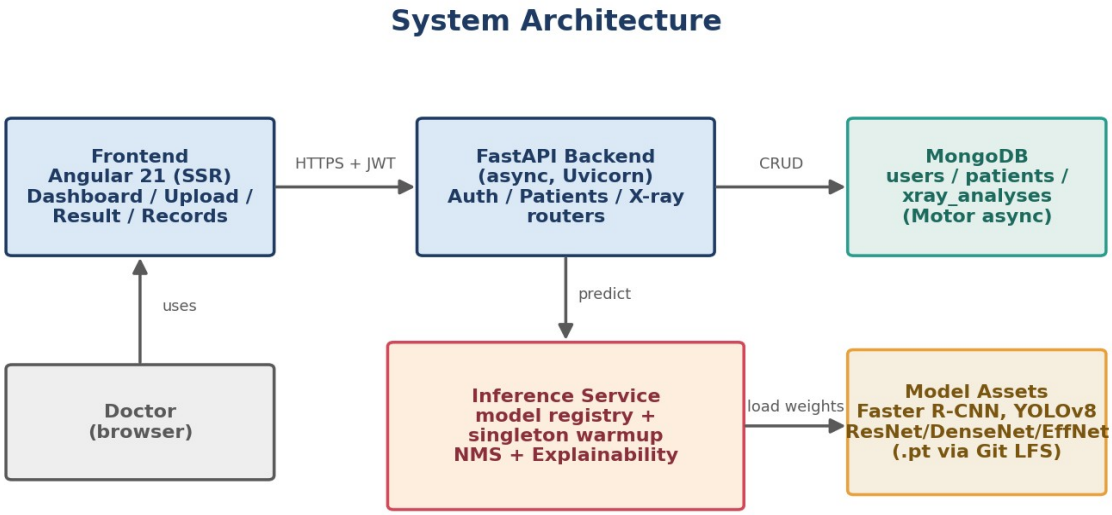

Figure B1. The three-tier architecture plus the inference service.

B1. The frontend (Angular 21)

The frontend is a single-page application built with Angular, written in TypeScript. Single-page means the browser loads one app and then swaps views without full page reloads, which feels fast. Its parts:

- **Components / pages:** login, signup, dashboard, upload, processing, result, patient records, profile, and settings. Each is a self-contained piece of UI.
- **Routing and guards:** the router maps URLs to pages; route guards block protected pages (like the dashboard) unless the user is logged in, and redirect logged-in users away from the login page.
- **State service:** a central analysis-state service holds the current analysis and history so different pages share data without passing it around manually.
- **API service:** one service centralizes every call to the backend, so the rest of the app never deals with HTTP directly. It attaches the login token to each request.
- **Server-side rendering (SSR):** public pages are pre-rendered on a small Express server for a fast first paint and better link previews; the app then 'hydrates' into the interactive single-page app.

B2. The backend (FastAPI)

The backend is a FastAPI service in Python. FastAPI is asynchronous, meaning it can handle many requests at once without blocking: while one request waits on the database or the model, the server serves others. Its parts:

- **Routers:** the API is split by resource into auth, patients, and x-ray routers, which keeps the code organized.

- **Pydantic models:** every request and response is validated against a typed schema, so malformed data is rejected automatically with a clear error.
- **Lifespan startup:** when the server boots it connects to MongoDB and warms up the default model (loads it into memory once), then cleans up on shutdown.
- **Model warmup (singleton):** the model is loaded a single time at startup and reused for every request, so users never pay the multi-second load cost. This is the key performance decision.

B3. The inference service, step by step

This is the heart of the backend. When an image arrives, it:

- **Validates** the file: allowed type (jpg/png/etc.), size under 10 MB, and that it actually decodes as an image.
- **Looks up the model** in the model registry (a table mapping a model key to its weights, task, and thresholds).
- **Runs the model** to get raw predictions (boxes and scores, or a class probability).
- **Applies Non-Maximum Suppression (NMS):** detectors often output several overlapping boxes for one finding; NMS keeps the highest-confidence box and drops others that overlap it by more than 30 percent, so the doctor sees one clean box per region.
- **Renders** the annotated image and an explainability heatmap.
- **Returns** a structured result: decision, boxes, confidence, processing time, and image paths, which the backend then saves.

Confidence thresholds: a detection above 0.10 is shown as 'suspected'; above 0.25 it is treated as a confident finding. These thresholds are configurable per model in the registry.

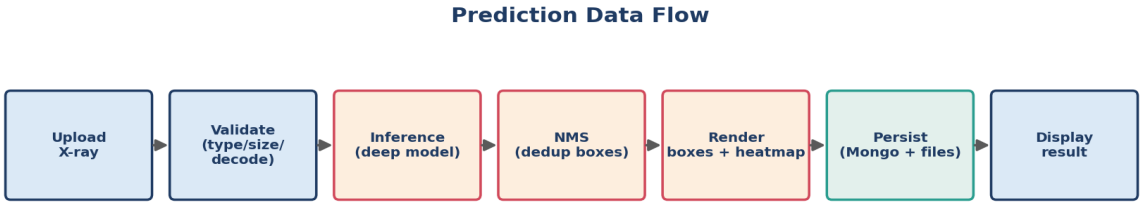

Figure B2. The prediction data flow.

B4. Security

- **JWT authentication:** on login the server issues a signed JSON Web Token (a tamper-proof ticket with an expiry). The browser sends it on every request; the server verifies the signature to know who you are, without storing sessions.
- **bcrypt password hashing:** passwords are never stored in plaintext. bcrypt adds a random salt and is deliberately slow, which makes stolen password databases very hard to crack.
- **Per-user authorization:** every database query is filtered by the logged-in doctor's id, so one doctor can never read another's patients. This is tested.
- **Input validation and CORS:** uploads are checked before they reach a model, and the API only accepts requests from known frontend origins.
- **Production secret guard:** the server refuses to start in production with the default secret key, preventing a common, dangerous deployment mistake.

B5. The database (MongoDB)

We use MongoDB, a document database, through the asynchronous Motor driver. It has three collections: users, patients, and xray_analyses. We chose a document database because one analysis is naturally one self-contained document (its boxes, scores, heatmap paths, and timestamps are always read together), which maps cleanly to a JSON-like document and avoids complex joins. Unique indexes enforce that emails and identifiers are not duplicated, and a compound index on owner plus creation time makes loading a doctor's history fast.

B6. The API

Endpoint	Method	What it does
/api/auth/signup, /login, /me	POST/GET	Register, log in (returns a token), get current user
/api/patients	CRUD	Create, read, update, delete patients (per doctor)
/api/xray/upload	POST	Upload an image, run inference, save the result
/api/xray, /api/xray/{id}	GET/DELETE	List or manage saved analyses
/health, /docs	GET	Health check; interactive Swagger documentation

B7. A full request, traced end to end

1) The doctor logs in; the frontend stores the JWT. 2) They open a patient and upload an X-ray; the upload page shows a preview. 3) The frontend's API service POSTs the image (with the token) to /api/xray/upload. 4) FastAPI verifies the token, validates the file, and hands it to the inference service. 5) The warmed model predicts; NMS cleans the boxes; a heatmap is rendered. 6) The result is written to the xray_analyses collection and the annotated image is saved. 7) The backend returns the structured result; the frontend shows it on the result screen and adds it to the patient's history.

B8. Deployment and configuration

- **Configuration** lives in environment variables (the database URL, the secret key, allowed origins, token expiry), kept out of version control.
- **Model weights** are stored with Git LFS because they are large binary files, so the repository stays small and clones fetch them on demand.
- **Scaling:** the API keeps no per-user state in memory (all state is in the database and the token), so it can be cloned behind a load balancer; the model can also be moved to its own service for heavy load.
- **Running it:** start MongoDB, run the backend with Uvicorn, build and serve the Angular frontend; full steps are in the README and the thesis user guide.

B9. Section B - Questions the examiners may ask

Q: Why FastAPI and async?

A: Inference and database calls involve waiting; an async server keeps serving other users during that wait, so the app stays responsive under load without a thread per request.

Q: Why MongoDB instead of a SQL database?

A: An analysis is one self-contained nested document that we always read as a whole, which fits a document store naturally and avoids multi-table joins. We still enforce structure with indexes and validation.

Q: How does login actually work?

A: On login we verify the bcrypt-hashed password and issue a signed JWT with an expiry. The browser sends that token on each request; the server verifies the signature to identify the user without server-side sessions.

Q: How do you stop one doctor seeing another's patients?

A: Every query is filtered by the authenticated doctor's id, so records that are not theirs simply return not-found. We have an automated test that proves cross-user access is rejected.

Q: Why load the model once at startup?

A: Loading a model takes seconds; doing it per request would make the app slow. We load it once (warmup) and reuse it, so each prediction is fast and memory use is predictable.

Q: What is NMS and why do you need it?

A: Non-Maximum Suppression removes duplicate overlapping detection boxes, keeping the most confident one per region (here, when overlap exceeds 30 percent), so the doctor sees one clean box instead of several.

Q: How do you keep uploads safe?

A: We validate type, size, and that the file decodes as an image before it reaches a model; the API is behind authentication and CORS; and secrets are kept in environment variables, not in code.

Q: How would you scale this to a hospital?

A: The API is stateless, so we run several copies behind a load balancer, move the model to a dedicated inference service or GPU server, and use a managed MongoDB cluster. Containerization and monitoring are the immediate next steps.